

you're currently in. If it doesn't find it there, it will check the folders listed in the `PATH` environment variable.

For example, on my Windows computer, the *python.exe* program file is located in the *C:\Users\Al\AppData\Local\Programs\Python\Python38* folder. To run it, I have to enter

C:\Users\Al\AppData\Local\Programs\Python\Python38\python.exe, or switch to that folder first and then enter *python.exe*.

This lengthy pathname requires a lot of typing, so instead I add this folder to the `PATH` environment variable. Then, when I enter *python.exe*, the command line searches for a program with this name in the folders listed in `PATH`, saving me from having to type the entire file path.

Because environment variables can contain only a single string value, adding multiple folder names to the `PATH` environment variable requires using a special format. On Windows, semicolons separate the folder names. You can view the current `PATH` value with the `path` command:

```
C:\Users\Al>path
C:\Path;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;
--snip--
C:\Users\Al\AppData\Local\Microsoft\WindowsApps
```

On macOS and Linux, colons separate the folder names:

```
al@ubuntu:~$ echo $PATH
/home/al/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games:/snap/bin
```

The order of the folder names is important. If I have two files named *someProgram.exe* in *C:\WINDOWS\system32* and *C:\WINDOWS*, entering *someProgram.exe* will run the program in *C:\WINDOWS\system32* because that folder appears first in the `PATH` environment variable.

If a program or command you enter doesn't exist in the `cwd` or any of the directories listed in `PATH`, the command line will give you an error, such as `command not found` or `not recognized as an internal or external command`. If you didn't make a typo, check which folder contains the program and see if it appears in the `PATH` environment variable.